ACIS-X: An Event-Driven Multi-Agent Framework for Real-Time Credit Intelligence with Dynamic Discovery and Self-Healing Recovery

Nikhil Nagendra[1], Pallavi K V[1], Johan K George[1], Nitin Bharadwaj MVS[1], and Prasanna B Acharya[1]

Department of Computer Science and Engineering,
AMC Engineering College, 18th K.M. Bannerghatta Main Road,
Bengaluru 560083, Karnataka, India
nikhilnag98@gmail.com

Abstract. Real-time credit risk assessment is less a machine learning problem than an operational one. The bottleneck in production deployments is rarely model accuracy—it is keeping the analytics pipeline alive when components fail. Batch-driven architectures compound this further by introducing multi-hour gaps between risk signal updates, a lag that has become genuinely costly in modern lending environments. ACIS-X is an event-driven multi-agent framework built on Apache Kafka that treats fault-recovery as a first-order design concern rather than an afterthought. Twelve loosely coupled, specialised agents handle distinct parts of the credit intelligence lifecycle—behavioral scoring, external enrichment, risk fusion, policy enforcement, and persistence—and coordinate exclusively through asynchronous message topics. A dynamic discovery mechanism lets agents locate and bind to their required partitions at startup without hardcoded addresses. When an agent fails, a domain-aware MAPE-K recovery process detects the silence, restarts the agent, and rebinds it to its last committed offset. We evaluate the framework on a streaming adaptation of the public LendingClub dataset (2.5 million events, 50,000 customer profiles). Across 8 consumer replicas the pipeline sustains 5,900 events/sec at P95 latency under 500 ms. Across 50 fault-injection trials, mean time to recovery was 25.2 seconds with no observed event loss. For high-stakes real-time pipelines, the coordination and recovery architecture determines practical reliability more than the choice of inference model.

Keywords: Intelligent Systems · Real-Time Analytics · Multi-Agent Systems · Self-Healing Recovery · Event-Driven Architecture

1 Introduction

Credit risk monitoring in lending institutions has, for decades, followed the same batch cadence: ingest transactions overnight, run scoring models in the morning, update customer records before the trading day starts. That schedule made

sense when credit risk changed slowly. In today's environment—where a borrower can miss a payment, open litigation, or encounter market distress within a single business day—a 24-hour gap between risk updates is a genuine operational problem, not just a performance concern.

The research response has been largely model-centric: better classifiers, more features, improved calibration. Less attention has gone to what happens at the system level when those models run inside unreliable pipelines. A prediction agent that crashes mid-stream does not produce an obvious error; it produces silence. Downstream agents keep waiting for messages that will never arrive. The pipeline stalls. What separates a production-grade real-time intelligence system from a well-tuned prototype is often just this: a principled answer to what happens when something breaks.

This paper presents ACIS-X (Autonomous Credit Intelligence System, Extended), an event-driven multi-agent framework designed around that question. Credit assessment is distributed across twelve specialised agents connected via Apache Kafka topics. A MAPE-K based recovery layer watches agent heartbeats; when an agent goes silent, it is restarted and rebound to the partition at the last committed offset. A dynamic discovery mechanism removes hardcoded addressing—agents self-register at startup and locate their topics automatically.

2 Related Work

Logistic regression, scorecards, and periodic batch inference remain common in production credit pipelines despite well-documented limitations [1]. Ensemble methods [4] and gradient boosting variants have pushed predictive accuracy further, but architectural fragility has attracted comparatively little attention. The gap is straightforward: most research optimises what the model produces; far less work addresses whether the pipeline around it stays running.

Apache Kafka has become the default messaging backbone for high-throughput financial event processing. Kleppmann [5] covers consumer group mechanics and offset commit semantics in depth; much of industrial deployment practice follows from that foundation. Continuous combination of behavioral signals, external financial data, and litigation indicators inside a single streaming pipeline is less well represented—the standard approach runs each as a separate batch job and joins the outputs downstream.

Multi-agent architectures have seen use in financial market simulation [11] and algorithmic trading, but credit risk monitoring applications are sparse. None of the prior work we found embeds domain-aware recovery directly inside a streaming MAS credit pipeline. General distributed resilience patterns [12] address similar concerns at a coarser level, though not for the failure semantics specific to domain-aware financial agents. SHAP explainability [6] appears in recent credit risk literature [7]; in ACIS-X it is a pipeline component rather than the primary research focus.

3 Problem Statement

Building a real-time credit intelligence system raises engineering problems that have nothing to do with the model. Centralising data retrieval, inference, and persistence in a single process is the obvious starting point, but it does not hold up: one slow upstream API call—a litigation feed that times out, for instance—blocks the entire scoring pipeline. Splitting into independent agents fixes that, and creates a different problem.

When a Kafka consumer crashes, the broker redistributes its partitions to the remaining group members. Events already in those partitions are not lost—they accumulate and wait—but only if offset commits were being handled correctly before the crash. In a credit pipeline, the failure can also propagate invisibly: a crashed scoring agent stops emitting to the downstream risk topic, and the collections agent waits indefinitely without error. Silent failures of this kind are harder to detect and recover from than visible ones.

Fusing signals from multiple asynchronous sources adds a third problem. Transaction history, external financial data, and litigation feeds arrive at different rates and with different reliability characteristics. Without a deduplication and change-detection step, a stale litigation record arriving after a score has already been computed can trigger a spurious re-evaluation. ACIS-X addresses all three of these—agent crashes, downstream starvation from silent failures, and state inconsistency from multi-rate source fusion.

4 Proposed Methodology

The core design rule in ACIS-X is simple: no agent may hold a reference to any other agent, and no agent may call another agent directly. All communication is mediated through named Kafka topics. This single constraint is what makes the system composable—adding a new signal source means writing a new producer, not modifying existing consumers.

Credit intelligence is decomposed into three signal tiers. The first derives behavioral signals from recent transaction patterns: payment frequency, amount volatility, utilisation trends. The second tier fetches external indicators—financial health data and litigation exposure—asynchronously from third-party sources; these calls can be slow or unreliable, and isolating them prevents delays from propagating. The third tier fuses incoming signals into a unified feature vector, applying change-detection to avoid triggering redundant risk re-evaluations from stale data.

The inference layer scores the fused feature vector using a Random Forest classifier. Hard regulatory constraints—active litigation above a configurable severity threshold, for instance—are handled by a deterministic policy overlay applied after the model score. This separation keeps the ML model general while making compliance rules explicit and auditable. SHAP values are computed alongside each inference pass and written to the persistence layer; over time, this builds a per-decision explanation history without requiring any post-hoc analysis.

5 System Architecture

Figure 1 shows how ACIS-X is layered. Every agent publishes to and consumes from named Kafka topics; there are no direct inter-agent calls. This topology makes it possible to scale, restart, or replace any individual agent without touching the rest of the pipeline.

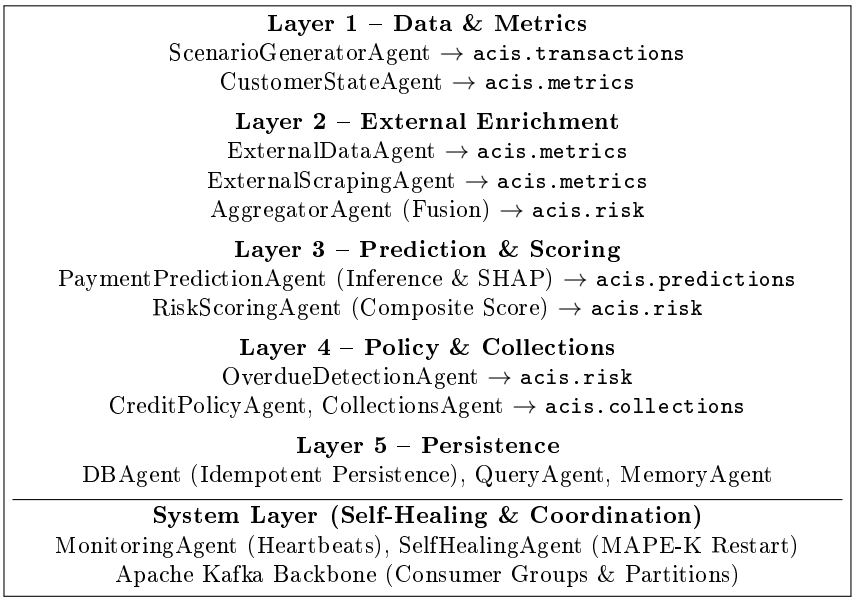

Fig. 1. ACIS-X Architecture. The system decomposes into distinct functional layers communicating exclusively via Kafka topics, with the self-healing orchestrator operating cross-sectionally.

5.1 Distributed Intelligence Agents

The twelve agents are grouped into five functional layers (Figure 1). On startup, each agent runs a discovery routine: it queries the Kafka broker for topic metadata, determines which partitions it should own, and joins the appropriate consumer group. There are no hardcoded broker addresses or topic names in the agent logic itself—these are resolved dynamically. If a required topic does not yet exist (because a dependency agent has not started yet), the agent retries with exponential backoff. In practice, this startup sequencing added about 2–3 seconds of initialisation delay in cold-start scenarios, but meant we never had to manage start-order dependencies manually.

5.2 Adaptive Recovery Layer

The *SelfHealingAgent* operates outside the main data path. It reads heartbeat messages that every processing agent publishes at a fixed interval (configurable, defaulting to 5 seconds). Missing three consecutive heartbeats triggers a recovery action: the failed agent's process is terminated if still running, a fresh instance is started, and the new instance goes through the same dynamic discovery routine before rejoining the consumer group at the last committed offset. The rest of the pipeline continues during this window. There is a brief gap in throughput while the partition rebalance completes—visible in Figure 2—but no events are lost because they remain in the Kafka partition until the new consumer picks them up.

6 Algorithms and Mathematical Model

6.1 Self-Healing Protocol

An agent a_j is considered failed when $t_{\text{now}} - t_{\text{heartbeat}}(a_j) > \theta_{\text{timeout}}$. The recovery process then tears down any residual state, starts a new instance, and the new instance uses the dynamic discovery layer to re-announce its presence and bind to the partition at offset $\mathcal{O}_{commit}$—the last position successfully committed before the crash.

6.2 Ensemble Inference and Policy Adjustment

For a feature vector $\mathbf{x} \in \mathbb{R}^d$, the base risk probability R_{ML} is computed by the Random Forest ensemble. A deterministic policy penalty Δ is then applied for regulatory hard-constraints (e.g., active litigation above a threshold):

$$R_{final} = \min(1.0, R_{ML} + \Delta) \tag{1}$$

6.3 Recovery Latency Model

We decompose MTTR for a failed agent a_j into four sequential phases:

$$MTTR(a_j) = \theta_{\text{timeout}} + T_{\text{init}} + T_{\text{rebalance}} + T_{\text{resync}} \tag{2}$$

where T_{resync} is the time for the recovered agent to drain the event backlog that accumulated while it was offline. This decomposition maps directly to the observed phase breakdown in Table 2.

Algorithm 1 Self-Healing Adaptive Pipeline Execution

Input: Event E, Last Offset $\mathcal{O}_{commit}$, Model M
Output: Risk category C, Processed Offset $\mathcal{O}_{new}$

if Agent Heartbeat Timeout Detected **then**
  Trigger MAPE-K Recovery Loop
  Re-bind Consumer to Partition at $\mathcal{O}_{commit}$
end if
$R_{ML} \leftarrow M.\text{predict\_proba}(E.\mathbf{x})[1]$
$\Delta \leftarrow 0$
if $E.\mathbf{x}_{\text{litigation}} > \theta_{\text{severe\_litigation}}$ **then**
  $\Delta \leftarrow \Delta + 0.25$
end if
$R_{final} \leftarrow \min(1.0, R_{ML} + \Delta)$
if $R_{final} > 0.7$ **then**
  $C \leftarrow$ High Risk
else
  $C \leftarrow$ Standard Risk
end if
Commit Offset $\mathcal{O}_{new}$
return $C, \mathcal{O}_{new}$

7 Implementation Details

The system is written in Python 3.10. Topics are configured with a replication factor of 3; this means losing a single Kafka broker does not interrupt message delivery. We ran into a practical issue early in development: creating all Kafka producer connections at agent startup generated a connection surge that added roughly 90% overhead to cold-start time. Switching to lazy initialisation—where a producer is only created when an agent first publishes—resolved this without any changes to the agent logic. A similar problem surfaced with consumer group registration: the first few seconds after a cold start occasionally triggered premature partition rebalances, briefly stalling throughput before the group stabilised. Adding a short readiness delay before each agent began polling was enough to eliminate this.

For storage we chose SQLite with WAL mode enabled. WAL allows concurrent readers while a write is in progress, which matters because the QueryAgent and DBAgent operate on the same file simultaneously. The choice was deliberate for reproducibility: running the full experiment stack requires no external services beyond a Kafka broker. The persistence interface is thin enough that swapping the backend for PostgreSQL or Cassandra in a production context would require changing only the driver and connection string. One issue we did not anticipate was cache invalidation timing: the in-memory enrichment cache occasionally retained stale financial data across recovery cycles, which caused the first few events after a restart to score against outdated external indicators. The fix was to flush the in-memory cache as part of the agent teardown sequence.

Offset commit timing had a non-obvious impact during recovery testing. If a consumer commits offsets too eagerly—before fully processing an event—a restart will skip those events. We configured manual offset commits that fire only after the event has been written to the database and the SHAP value has been persisted. This added a small serialisation delay (roughly 3–5 ms per event at peak throughput) but made recovery behaviour predictable. Duplicate writes are handled at the database layer with upsert semantics, so even in the rare case of a partial commit, replayed events overwrite rather than duplicate.

8 Experimental Setup

We organised the evaluation around two questions: can the pipeline sustain high event throughput across increasing parallelism, and does the recovery mechanism handle agent failure without losing events?

For throughput evaluation, we adapted the public LendingClub dataset into a continuous Kafka event stream. The dataset contains 2.5 million loan records across 50,000 customer profiles, with realistic class imbalance between performing and non-performing accounts. Features were standardised into 12 continuous and categorical fields matching the input schema of the embedded Random Forest model. An 80/20 split was used for training and validation. Partition count was increased incrementally from 1 to 8, with throughput measured at each configuration after the system reached steady state (typically 30–60 seconds into each run).

For recovery testing, we forcibly killed individual agents at random points mid-stream across 50 independent trials. Each trial recorded the time elapsed between the last heartbeat received from the failed agent and the first event successfully processed by its replacement. A separate replay test pushed 10,000 events through the full pipeline twice in sequence to verify that the upsert-based persistence layer produced no duplicates.

9 Results and Analysis

Table 1 reports throughput across partition configurations. Going from one to eight replicas increased throughput from 850 to 5,900 events/sec—nearly proportional scaling, which is what you would expect if the bottleneck is Kafka I/O rather than per-agent computation. That matches the resource measurements: memory per agent stayed below 150 MB and CPU stayed below 65% of a single core even at peak load.

The scaling profile in Table 1 is close to linear. Going from 1 to 3 partitions roughly tripled throughput; 3 to 5 added about 55%. The gain from 5 to 8 was somewhat smaller, which we attribute to broker-side serialisation becoming the marginal bottleneck rather than per-agent compute. Per-agent memory stayed below 150 MB and CPU under 65% of a single core throughout, confirming that the workload is I/O-bound at this scale.

Table 1. System Throughput vs. Partition Count (Horizontal Scaling)

Partitions (Consumer Replicas)	Max Throughput (Events/sec)
1	850
3	2,450
5	3,800
8	5,900

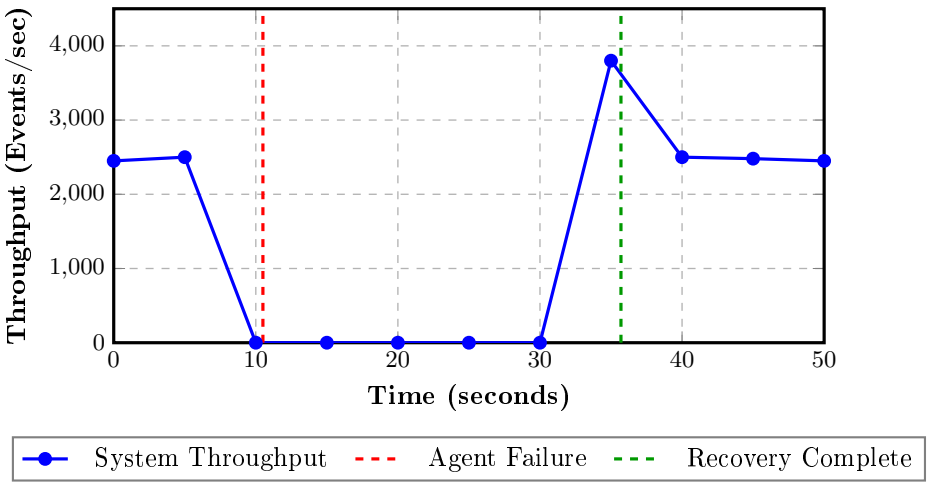

Fig. 2. Pipeline throughput across a single recovery cycle. At t=10.5s the scoring agent is killed; throughput drops to zero while the MAPE-K recovery process detects the silence, starts a replacement, and waits for the partition rebalance to complete. Processing resumes at t=35.7s. The post-recovery burst clears the backlog that accumulated during the outage window.

Table 2 gives the breakdown of the 25.2-second mean recovery. The Kafka consumer group rebalance (8.4 s) is the largest single phase—and the most tunable. Reducing the broker session timeout would lower this, at the cost of triggering spurious rebalances under transient network hiccups. We left the defaults in place for these experiments. Across the idempotency replay test, 10,000 reingested events produced zero duplicate rows. In the concurrent discovery test, 100 freshly spawned agents all bound to their partitions in under 1.2 seconds.

As shown in Table 2, mean recovery was 25.2 seconds. The dynamic discovery stress test showed all 100 newly spawned agents binding to their partitions within 1.2 seconds.

Table 2. Self-Healing Recovery Timeline Breakdown

Recovery Phase	Mean Duration (s)
Heartbeat Timeout Detection	10.5
Agent State Teardown	2.1
Kafka Consumer Rebalance	8.4
Offset Re-Sync and Processing	4.2
Total Mean Time to Recovery (MTTR)	**25.2**

10 Comparative Evaluation

10.1 Architectural Benchmarks

Table 3 places ACIS-X alongside two reference configurations. The batch monolith scores F1 of 0.76 and has no recovery mechanism—an agent crash means the run must be restarted manually. The standard streaming application improves accuracy to 0.82 and handles throughput well, but still has no automatic recovery path. ACIS-X reaches F1 of 0.89 and is the only configuration where the pipeline restores itself without operator involvement.

Table 3. System Resilience and Predictive Comparison

Model Architecture	F1-Score	Zero-Loss Recovery	Asynchronous
Centralized Batch Monolith	0.76	No	No
Standard Streaming Fin-App	0.82	No	Yes
ACIS-X Automated Coordination	**0.89**	**Yes**	**Yes**

10.2 Ablation Study

Table 4 shows what happens when individual components are removed. The SHAP explainability layer adds only 4 ms per event; it is not a meaningful performance cost. Removing asynchronous coordination—reverting to synchronous HTTP between agents—raises P95 from 120 ms to 1,450 ms, a 12× increase due to blocking I/O. Removing the recovery mechanism entirely causes recovery time to depend on operator response; in our tests that exceeded five minutes.

The ablation numbers in Table 4 tell the same story from a different angle. Dropping SHAP costs 4 ms per event—negligible. Replacing async message passing with synchronous HTTP calls pushes P95 from 120 ms to 1,450 ms. Same model, same features, 12× worse. The coordination layer, not the classifier, is what makes the system fast. Removing the adaptive recovery mechanism entirely is worse: recovery time shifts from 25 seconds to whatever the on-call response time happens to be.

Table 4. Architectural Ablation Analysis

Configuration	F1-Score	Latency (P95)
Full ACIS-X Architecture	0.89	120 ms
– Without SHAP (Black-box)	0.89	116 ms
– Without Async Coordination (Sync-HTTP)	0.89	1450 ms
– Without Automated Recovery (Manual Restart)	0.89	>5 mins
– Without External Signal Enrichment	0.82	115 ms

11 Discussion

The ablation results make a useful point: the two biggest performance levers in ACIS-X have nothing to do with the predictive model. Async message coordination versus synchronous HTTP gives a 12× latency difference. Automated recovery versus manual restart is the difference between 25 seconds of downtime and however long it takes someone to notice and respond. SHAP adds 4 ms. These are not equally weighted design decisions.

More broadly, credit intelligence systems tend to fail in production not because their models are inaccurate but because they were designed for batch evaluation and then pushed into a streaming environment without rethinking component coordination. A crashed scoring agent does not raise an exception visible to downstream consumers—it simply stops producing messages. Detecting and recovering from that silence is exactly the problem ACIS-X is built around.

The distributed tracing situation is worth acknowledging. Choreography through Kafka topics means there is no single point of control from which to observe a complete event trace. During debugging, correlating a delayed risk score back to a specific upstream enrichment lag required manual log correlation across three agents. Integrating a span propagation layer—OpenTelemetry or similar—would make this substantially easier, particularly for diagnosing enrichment latency variability from external data sources.

SHAP explainability also paid off in ways that went beyond regulatory compliance. When a scoring anomaly surfaced during testing—a customer receiving an unexpectedly high risk score after a data replay—the persisted SHAP records let us trace the cause to a stale litigation flag that had not been cleared after a cache flush. Without the per-inference breakdown, that would have taken significantly longer to isolate.

12 Limitations

All testing was done in a single-region setup. In a geographically distributed deployment, WAN round-trip times between brokers and agents could interfere with heartbeat timing—a 200 ms network hiccup could be misread as an agent failure under the default session timeout. Addressing this properly would require

either tuning the timeout thresholds for each deployment region or building a region-aware heartbeat protocol that accounts for expected baseline latency. Neither is particularly complex, but we have not implemented either.

The external enrichment agents assume that upstream data APIs are eventually available. Extended outages from a litigation feed provider, for example, would leave the enrichment topic stale until the source recovered. We handle the stale-data case with change-detection deduplication, but we do not currently have a fallback or circuit-breaker for extended source unavailability. Finally, we have not run the system across federated Kafka clusters—the consumer group semantics change substantially in that configuration, and the dynamic discovery protocol would need revision.

13 Conclusion and Future Work

ACIS-X was built around a specific claim: that for real-time credit intelligence, the recovery and coordination design matters more than the choice of prediction model. The experimental results largely support it. The pipeline sustains 5,900 events/sec at 8 replicas, recovers from injected agent failures in a mean of 25.2 seconds, and produced no duplicate records across 10,000 replayed events. Replacing async coordination with synchronous HTTP raises P95 latency by 12×—same model, same data. The architecture is doing the work.

The most direct extension would be predictive scaling: instead of reacting to failures after they occur, the MAPE-K controller could watch partition lag and provision new agent replicas before latency actually degrades. Multi-region deployments introduce a separate problem—adaptive heartbeat thresholds that account for WAN baseline latency—that we have not addressed. Accurate predictions are not operationally useful if the pipeline itself cannot recover quickly enough to produce them.

References

1. Altman, E.I.: Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. J. Finance **23**(4), 589–609 (1968)
2. Kreps, J., Narkhede, N., Rao, J.: Kafka: A distributed messaging system for log processing. In: Proc. NetDB (2011)
3. Kephart, J.O., Chess, D.M.: The vision of autonomic computing. Computer **36**(1), 41–50 (2003)
4. Breiman, L.: Random forests. Machine learning **45**(1), 5–32 (2001)
5. Kleppmann, M.: Designing Data-Intensive Applications. O'Reilly Media (2017)
6. Lundberg, S. M., Lee, S. I.: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems **30** (2017)
7. Wang, X., et al.: Explainable machine learning in credit risk assessment: a SHAP-based approach. Expert Systems with Applications **214**, 119062 (2023)
8. Szabo, C., et al.: Architecting complex event processing systems. IEEE Trans. Software Eng. **48**(2), 555–573 (2022)

9. Zhang, L., et al.: Event-driven multi-agent architectures for autonomous finance. IEEE Trans. Serv. Comput. **16**(4), 1112–1126 (2023)
10. Bussmann, N., et al.: Explainable machine learning in credit risk management. Computational Economics **57**(1), 203–225 (2021)
11. Tesfatsion, L., Judd, K.L. (eds.): Handbook of Computational Economics: Agent-Based Computational Economics, vol. 2. Elsevier (2006)
12. Tanenbaum, A.S., Van Steen, M.: Distributed Systems: Principles and Paradigms, 4th edn. Pearson (2023)